MATERIAL DIDÁCTICO · PMDM

RELACIÓN 2

Modelos y RecyclerView

APRENDER PROGRAMANDO, PASO A PASO

PROGRAMACIÓN MULTIMEDIA Y DISPOSITIVOS MÓVILES · 2.º DAM

ENUNCIADOS · COMPROBACIONES · AMPLIACIONES

Guía del alumnado

Esta relación contiene 4 aplicaciones progresivas. Debes construirlas y comprobar resultados, no únicamente conseguir que compilen.

Qué debes haber hecho antes

Tutorial 2: inventario con RecyclerView, data class y un Adapter. Es obligatorio haber realizado la Relación 1.

Condiciones generales

Crea un proyecto Android Views por ejercicio. Incluye modelo, Adapter y XML de fila en archivos independientes. Añade la dependencia RecyclerView 1.4.0 en app/build.gradle.kts (compileSdk 35 o posterior). No se exige guardar datos todavía.

- Entrega cada proyecto completo de Android Studio, no únicamente un archivo MainActivity.kt.
- Mantén cada función de negocio separada de los widgets: recibe datos y devuelve el resultado.
- No utilices **!!** ni conversiones inseguras para los datos escritos por el usuario.
- Incluye al menos una captura de caso correcto y otra de dato inválido por ejercicio.

Relación de aplicaciones

N.º	Aplicación	Núcleo de trabajo
01	Videoteca del festival	RecyclerView, filtros y modelo de película
02	Pedidos del mercadillo	listas de objetos y totales monetarios
03	Clasificación de equipos	ordenar objetos y notificación al adaptador
04	Biblioteca con préstamos	RecyclerView, interacción por fila, copy

> **Uso de las bases:** la carpeta **bases** contiene una pantalla de partida opcional de cada ejercicio. No es un proyecto Gradle: crea la app con Empty Views Activity y copia los archivos al proyecto. Deberás programar toda la solución.

EJERCICIO 01 / RECYCLERVIEW, FILTROS Y MODELO DE PELÍCULA

01. Videoteca del festival

Diseña una videoteca con filas de dos TextView: una línea de título/género y otra con duración. Debe permitir mostrar toda la lista o solo cortos.

Elementos mínimos de la pantalla

etTitulo (EditText), **etGenero** (EditText), **etMinutos** (EditText (entero)), **btnAgregar** (Button), **btnFiltrar** (Button), **tvResumen** (TextView), **rvPeliculas** (RecyclerView).

Requisitos obligatorios

1. Usa **data class Pelicula(titulo, genero, minutos)**, con minutos entre 1 y 300.
2. Crea **item_pelicula.xml** y **PeliculaAdapter.kt** con ViewHolder y tres métodos del adaptador.
3. El formulario añade películas válidas y actualiza RecyclerView correctamente.
4. Un botón alterna entre «Todos» y «Cortos» (≤ 30 minutos), sin borrar la lista completa.
5. El resumen distingue total de registros y visibles. Las funciones de cálculo no accederán a los widgets.

Casos que debes comprobar

Entrada o acción	Resultado esperado
Corto 20 min + Largo 100 min	2 registros visibles inicialmente
Filtrar cortos	Solo 1 fila visible
Volver a todos	2 filas
Minutos 0	No se añade

Ampliación voluntaria

Añade otro filtro por género sin duplicar adaptadores innecesariamente.

> **Evidencias:** pantalla con datos correctos, prueba de error y una explicación breve de las funciones de cálculo o gestión de datos.

EJERCICIO 02 / LISTAS DE OBJETOS Y TOTALES MONETARIOS

02. Pedidos del mercadillo

Cada pedido registra cliente, unidades y precio por unidad. RecyclerView muestra el importe calculado de cada pedido.

Elementos mínimos de la pantalla

etCliente (EditText), **etUnidades** (EditText (entero)), **etPrecio** (EditText (decimal)), **btnAgregar** (Button), **tvResumen** (TextView), **rvPedidos** (RecyclerView).

Requisitos obligatorios

1. Crea **Pedido** con cliente, unidades y precioUnitario.
2. Unidades 1..50; precio 0..200; cliente no vacío.
3. Define **importePedido(pedido: Pedido): Double** y otra función para el total de la colección.
4. Muestra en cada fila cliente, unidades e importe con dos decimales.
5. Permite eliminar una fila mediante un botón situado dentro de **item_pedido.xml** y callback del adaptador a la Activity.

Casos que debes comprobar

Entrada o acción	Resultado esperado
Ana / 3 / 2.50	7,50 €
Luis / 2 / 4.00	Total 15,50 €
Borrar Ana	Total 8,00 €
Unidades 0	Rechazado

Ampliación voluntaria

Si un cliente repite, agrupa sus pedidos solo en el resumen, sin modificar los registros.

> **Evidencias:** pantalla con datos correctos, prueba de error y una explicación breve de las funciones de cálculo o gestión de datos.

EJERCICIO 03 / ORDENAR OBJETOS Y NOTIFICACIÓN AL ADAPTADOR

03. Clasificación de equipos

Muestra una clasificación deportiva calculada a partir de victorias y empates, con filas de equipo y puntos.

Elementos mínimos de la pantalla

etEquipo (EditText), **etVictorias** (EditText (entero)), **etEmpates** (EditText (entero)), **btnAgregar** (Button), **btnOrdenar** (Button), **tvEstado** (TextView), **rvEquipos** (RecyclerView).

Requisitos obligatorios

1. Crea **Equipo** con nombre, victorias y empates (ambos 0..40).
2. Puntuación = 3 × victorias + empates.
3. No permitas equipos de nombre repetido.
4. Al pulsar Ordenar, muestra los equipos por puntos descendentes; a igualdad, por nombre ascendente.
5. La tabla se refresca y conserva las referencias correctas tras ordenar.

Casos que debes comprobar

Entrada o acción	Resultado esperado
Azul / 2 / 1	7 puntos
Rojo / 3 / 0	9 puntos
Ordenar	Rojo primero
azul repetido	Rechazado

Ampliación voluntaria

Añade la media de puntos por equipo y destaca el líder.

> **Evidencias:** pantalla con datos correctos, prueba de error y una explicación breve de las funciones de cálculo o gestión de datos.

EJERCICIO 04 / RECYCLERVIEW, INTERACCIÓN POR FILA, COPY

04. Biblioteca con préstamos

Construye un catálogo de libros que permite cambiar el estado prestado/disponible sin borrar registros.

Elementos mínimos de la pantalla

etIsbn (EditText), **etTitulo** (EditText), **btnAgregar** (Button), **tvResumen** (TextView), **rvLibros** (RecyclerView).

Requisitos obligatorios

1. Modela **Libro(isbn: String, titulo: String, disponible: Boolean)** en una data class.
2. ISBN y título obligatorios; no puede repetirse el ISBN. Los nuevos libros se crean disponibles.
3. Cada fila tiene botón «Prestar» o «Devolver» según el estado actual.
4. La Activity cambia el libro mediante **copy(disponible = !libro.disponible)** y notifica la posición modificada al Adapter.
5. Resumen: total de libros, disponibles y prestados. No desaparecen registros al cambiar su estado.

Casos que debes comprobar

Entrada o acción	Resultado esperado
ISBN A1 / El viaje	1 disponible
Prestar A1	0 disponibles, 1 prestado
Devolver A1	1 disponible
ISBN A1 repetido	Rechazado

Ampliación voluntaria

Ordena con disponibles primero; documenta cómo cambia la notificación cuando reordenas.

> **Evidencias:** pantalla con datos correctos, prueba de error y una explicación breve de las funciones de cálculo o gestión de datos.